고객을 세그먼테이션하자 [프로젝트] - 정영웅

11-2. 데이터 불러오기

데이터 살펴보기

- **테이블에 있는 10개의 행만 출력하기**

```
SELECT *
FROM `project-c2dbf631-206a-40e8-a71.modulabs_project.data`
LIMIT 10;
```

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	536365	85123A	WHITE HANGING HEART T-LIG...	6	2010-12-01 08:26:00 UTC	2.55	17850	United Kingdom
2	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
3	536365	84406B	CREAM CUPID HEARTS COAT H...	8	2010-12-01 08:26:00 UTC	2.75	17850	United Kingdom
4	536365	84029G	KNITTED UNION FLAG HOT WA...	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
5	536365	84029E	RED WOOLLY HOTTIE WHITE H...	6	2010-12-01 08:26:00 UTC	3.39	17850	United Kingdom
6	536365	22752	SET 7 BABUSHKA NESTING BO...	2	2010-12-01 08:26:00 UTC	7.65	17850	United Kingdom
7	536365	21730	GLASS STAR FROSTED T-LIGHT...	6	2010-12-01 08:26:00 UTC	4.25	17850	United Kingdom
8	536366	22633	HAND WARMER UNION JACK	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom
9	536366	22632	HAND WARMER RED POLKA DOT	6	2010-12-01 08:28:00 UTC	1.85	17850	United Kingdom

- **전체 데이터는 몇 행으로 구성되어 있는지 확인하기**

```
SELECT COUNT(*) AS total_rows
FROM `project-c2dbf631-206a-40e8-a71.modulabs_project.data`;
```

행	total_rows
1	541909

데이터 수 세기

- **COUNT 함수를 사용해서, 각 컬럼별 데이터 포인트의 수를 세어 보기**

```
SELECT
  COUNT(InvoiceNo) AS InvoiceNo_count,
  COUNT(StockCode) AS StockCode_count,
  COUNT(Description) AS Description_count,
  COUNT(Quantity) AS Quantity_count,
  COUNT(InvoiceDate) AS InvoiceDate_count,
  COUNT(UnitPrice) AS UnitPrice_count,
  COUNT(CustomerID) AS CustomerID_count,
  COUNT(Country) AS Country_count
FROM `project-c2dbf631-206a-40e8-a71.modulabs_project.da
ta`;
```

행	InvoiceNo_count	StockCode_count	Description_count	Quantity_count	InvoiceDate_count	UnitPrice_count	CustomerID_count	Country_count
1	541909	541909	540455	541909	541909	541909	406829	541909

11-4. 데이터 전처리 방법(1): 결측치 제거

컬럼 별 누락된 값의 비율 계산

- **각 컬럼 별 누락된 값의 비율을 계산**
  - **각 컬럼에 대해서 누락 값을 계산한 후, 계산된 누락 값을 UNION ALL을 통해 합치기**

```
SELECT 'InvoiceNo' AS column_name,
       (COUNT(*) - COUNT(InvoiceNo)) / COUNT(*) AS null_
ratio
FROM `project-c2dbf631-206a-40e8-a71.modulabs_project.da
ta`

UNION ALL

SELECT 'StockCode',
       (COUNT(*) - COUNT(StockCode)) / COUNT(*)
```

```
FROM `project-c2dbf631-206a-40e8-a71.modulabs_project.da
ta`

UNION ALL

SELECT 'Description',
       (COUNT(*) - COUNT(Description)) / COUNT(*)
FROM `project-c2dbf631-206a-40e8-a71.modulabs_project.da
ta`

UNION ALL

SELECT 'Quantity',
       (COUNT(*) - COUNT(Quantity)) / COUNT(*)
FROM `project-c2dbf631-206a-40e8-a71.modulabs_project.da
ta`

UNION ALL

SELECT 'InvoiceDate',
       (COUNT(*) - COUNT(InvoiceDate)) / COUNT(*)
FROM `project-c2dbf631-206a-40e8-a71.modulabs_project.da
ta`

UNION ALL

SELECT 'UnitPrice',
       (COUNT(*) - COUNT(UnitPrice)) / COUNT(*)
FROM `project-c2dbf631-206a-40e8-a71.modulabs_project.da
ta`

UNION ALL

SELECT 'CustomerID',
       (COUNT(*) - COUNT(CustomerID)) / COUNT(*)
FROM `project-c2dbf631-206a-40e8-a71.modulabs_project.da
ta`
```

```
UNION ALL

SELECT 'Country',
       (COUNT(*) - COUNT(Country)) / COUNT(*)
FROM `project-c2dbf631-206a-40e8-a71.modulabs_project.data`;
```

행	column_name	null_ratio
1	InvoiceNo	0.0
2	StockCode	0.0
3	Description	0.002683107311...
4	Quantity	0.0
5	InvoiceDate	0.0
6	UnitPrice	0.0
7	CustomerID	0.249266943342...
8	Country	0.0

누락 비율이 0보다 큰 컬럼은 Description과 CustomerID이므로,

해당 두 컬럼에 누락된 값이 존재한다.

결측치 처리 전략

- **`StockCode = '85123A'`의 `Description`을 추출하는 쿼리문을 작성하기**

```
SELECT DISTINCT Description
FROM `project-c2dbf631-206a-40e8-a71.modulabs_project.data`
WHERE StockCode = '85123A';
```

행	Description
1	CREAM HANGING HEART T-LIG...
2	WHITE HANGING HEART T-LIG...

결측치 제거(Description, CustomerID) 쿼리를 먼저 실행하면서 일부 데이터가 삭제되어,

예시에서 보이는 '?'와 'wrongly marked carton 22804' 값이 제거되었습니다.

결측치 처리

- **DELETE 구문을 사용하며, WHERE 절을 통해 데이터를 제거할 조건을 제시**

```
DELETE FROM `project-c2dbf631-206a-40e8-a71.modulabs_project.data`
WHERE Description IS NULL
   OR CustomerID IS NULL;
```

이 문으로 data의 행 0개가 삭제되었습니다.

실제 삭제된 행 수는 **135,050개**이며, 캡처 시점에는 이미 삭제가 완료되어 **0개로 표시되었습니다.**

11-5. 데이터 전처리(2): 중복값 처리

중복값 확인

- **중복된 행의 수를 세어보기**
  - **8개의 컬럼에 그룹 함수를 적용한 후, COUNT가 1보다 큰 데이터를 세어보기**

```
SELECT COUNT(*) AS duplicate_rows
FROM (
  SELECT 1
  FROM `modulabs_project.data`
  GROUP BY InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country
  HAVING COUNT(*) > 1
);
```

행	duplicate_rows
1	4837

중복값 처리

- **중복값을 제거하는 쿼리문 작성하기**
  - **CREATE OR REPLACE TABLE 구문을 활용하여 모든 컬럼(*)을 DISTINCT 한 데이터로 업데이트**

```
CREATE OR REPLACE TABLE `modulabs_project.data` AS
SELECT DISTINCT *
FROM `modulabs_project.data`
```

이 문으로 이름이 data인 테이블이 교체되었습니다.

11-6. 데이터 전처리(3): 오류값 처리

`InvoiceNo` 살펴보기

- **고유(unique)한 `InvoiceNo`의 개수를 출력하기**

```
SELECT COUNT(DISTINCT InvoiceNo) AS unique_invoice_count
FROM `modulabs_project.data`
```

행	unique_invoice_c...
1	22190

- **고유한 `InvoiceNo`를 앞에서부터 100개를 출력하기**

```
SELECT DISTINCT InvoiceNo
FROM modulabs_project.data
```

```
LIMIT 100
```

행	InvoiceNo
1	541431
2	C541433
3	537626
4	542237
5	549222
6	556201
7	562032
8	573511
9	581180
10	539318
11	541998
12	548955
13	568172

너무 길어서 이정도만 넣었습니다.

- **`InvoiceNo`가 'C'로 시작하는 행을 필터링 할 수 있는 쿼리문을 작성하기 (100행까지만 출력)**

```
SELECT *
FROM project_name.modulabs_project.data
WHERE # [[YOUR QUERY]]
LIMIT 100;
```

행	InvoiceNo
1	541431
2	C541433
3	537626
4	542237
5	549222
6	556201
7	562032
8	573511
9	581180
10	539318
11	541998
12	548955
13	568172
14	577609
15	543037

너무 길어서 이정도만 넣었습니다.

- **구매 건 상태가 `Canceled` 인 데이터의 비율(%) - 소수점 첫번째 자리까지**

```
SELECT
  ROUND(
    COUNTIF(InvoiceNo LIKE 'C%') / COUNT(*) * 100,
    1
  ) AS canceled_rate_percent
FROM `modulabs_project.data`
```

행	canceled_rate_pe...
1	2.2

`StockCode` 살펴보기

- **고유한 `StockCode`의 개수를 출력하기**

```
SELECT COUNT(DISTINCT StockCode) AS unique_stockcode_cou
nt
FROM `modulabs_project.data`
```

행	unique_stockcod...
1	3684

- **어떤 제품이 가장 많이 판매되었는지 보기 위하여 `StockCode` 별 등장 빈도를 출력하기**
  - 상위 10개의 제품들을 출력하기

```
SELECT
  StockCode,
  COUNT(*) AS frequency
FROM `modulabs_project.data`
GROUP BY StockCode
ORDER BY frequency DESC
```

행	StockCode	frequency
1	85123A	2065
2	22423	1894
3	85099B	1659
4	47566	1409
5	84879	1405
6	20725	1346
7	22720	1224
8	POST	1196
9	22197	1110
10	23203	1108

- `StockCode` **의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고**
  - **숫자가 0~1개인 값**들에는 어떤 코드들이 들어가 있는지 출력하기

```
SELECT DISTINCT StockCode, number_count
FROM (
  SELECT StockCode,
    LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode,
r'[0-9]', '')) AS number_count
  FROM project_name.modulabs_project.data
)
WHERE # [[YOUR QUERY]];
```

행	StockCode
1	23166
2	20780
3	22492
4	22497
5	84969
6	22726
7	22728
8	85116
9	84997C
10	22773
11	22729
12	84558A
13	84997B
14	21064

- `StockCode` **의 컬럼에 있던 값 중에서 숫자를 제외한 문자만 남기고 문자가 몇 자리 수 인지 세고**
  - **숫자가 0~1개인 값들을 가지고 있는 데이터 수는 전체 데이터 수 대비 몇 퍼센트**인지 구하기 (소수점 두 번째 자리까지)

```
SELECT
ROUND(
  COUNTIF(number_count <= 1) / COUNT(*) * 100
, 2) AS percent_rows
FROM (
  SELECT
    LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(StockCode,
r'[0-9]', '')) AS number_count
  FROM `modulabs_project.data`
);
```

행	percent_rows
1	0.48

- **제품과 관련되지 않은 거래 기록을 제거하기**

```
DELETE FROM `project-c2dbf631-206a-40e8-a71.modulabs_pro
ject.data`
WHERE StockCode IN (
  SELECT DISTINCT StockCode
  FROM (
    SELECT StockCode
    FROM `project-c2dbf631-206a-40e8-a71.modulabs_projec
t.data`
    WHERE LENGTH(StockCode) - LENGTH(REGEXP_REPLACE(Stoc
kCode, r'[0-9]', '')) <= 1
  )
);
```

이 문으로 data의 행 1,915개가 삭제되었습니다.

`Description` 살펴보기

- **고유한 Description 별 출현 빈도를 계산하고 상위 30개를 출력하기**

```
SELECT Description, COUNT(*) AS description_cnt
FROM project-c2dbf631-206a-40e8-a71.modulabs_project.dat
a
GROUP BY Description
ORDER BY description_cnt DESC
LIMIT 30
```

행	Description	description_cnt
23	LUNCH BAG APPLE DESIGN	890
24	SET OF 4 PANTRY JELLY MOUL...	890
25	BAKING SET 9 PIECE RETROSP...	885
26	JAM MAKING SET PRINTED	883
27	RECIPE BOX PANTRY YELLOW ...	883
28	LUNCH BAG WOODLAND	850
29	ROSES REGENCY TEACUP AND ...	844
30	VICTORIAN GLASS HANGING T-...	843

- **서비스 관련 정보를 포함하는 행들을 제거하기**

```
DELETE
FROM `project-c2dbf631-206a-40e8-a71`.modulabs_project.d
ata
WHERE Description LIKE '%POSTAGE%';
```

이 문으로 data의 행 0개가 삭제되었습니다.

앞서 제품과 관련되지 않은 거래 기록 제거로 인해 0으로 나왔습니다.

- **대소문자를 혼합하고 있는 데이터를 대문자로 표준화 하기**

```
CREATE OR REPLACE TABLE `project-c2dbf631-206a-40e8-a71
`.modulabs_project.data AS
SELECT
  * EXCEPT(Description),
  UPPER(Description) AS Description
FROM `project-c2dbf631-206a-40e8-a71`.modulabs_project.d
ata;
```

이 문으로 이름이 data인 테이블이 교체되었습니다.

`UnitPrice` 살펴보기

- `UnitPrice`의 최솟값, 최댓값, 평균을 구하기

```
SELECT
  MIN(UnitPrice) AS min_price,
  MAX(UnitPrice) AS max_price,
  AVG(UnitPrice) AS avg_price
FROM `project-c2dbf631-206a-40e8-a71`.modulabs_project.d
ata;
```

행	min_price	max_price	avg_price
1	0.0	649.5	2.907457172951...

- 단가가 0원인 거래의 개수, 구매 수량(`Quantity`)의 최솟값, 최댓값, 평균 구하기

```
SELECT
  COUNT(*) AS cnt_quantity,
  MIN(Quantity) AS min_quantity,
  MAX(Quantity) AS max_quantity,
```

```
  AVG(Quantity) AS avg_quantity
FROM `project-c2dbf631-206a-40e8-a71`.modulabs_project.data
WHERE UnitPrice = 0;
```

행	cnt_quantity	min_quantity	max_quantity	avg_quantity
1	33	1	12540	420.5151515151...

- `UnitPrice = 0` **를 제거하고 일관된 데이터셋을 유지하기**

```
CREATE OR REPLACE TABLE `project-c2dbf631-206a-40e8-a71`.modulabs_project.data AS
SELECT *
FROM `project-c2dbf631-206a-40e8-a71`.modulabs_project.data
WHERE UnitPrice > 0;
```

이 문으로 이름이 data인 테이블이 교체되었습니다.

Unitprice = 0은 실제 판매자 데이터가 아닐 가능성이 높아서 분석 정확도를 위해 제거합니다.

11-7. RFM 스코어

Recency

- `InvoiceDate` **컬럼을 연월일 자료형으로 변경하기**

```
SELECT DATE(InvoiceDate) AS InvoiceDay, *
FROM `project-c2dbf631-206a-40e8-a71`.modulabs_project.data;
```

행	InvoiceDay	InvoiceNo	StockCode	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Description
1	2011-01-18	541431	23166	74215	2011-01-18 10:01:00 UTC	1.04	12346	United Kingdom	MEDIUM CERAMIC T
2	2011-01-18	C541433	23166	-74215	2011-01-18 10:17:00 UTC	1.04	12346	United Kingdom	MEDIUM CERAMIC T
3	2010-12-07	537626	22729	4	2010-12-07 14:57:00 UTC	3.75	12347	Iceland	ALARM CLOCK BAKE
4	2010-12-07	537626	71477	12	2010-12-07 14:57:00 UTC	3.25	12347	Iceland	COLOUR GLASS. STA
5	2010-12-07	537626	20782	6	2010-12-07 14:57:00 UTC	5.49	12347	Iceland	CAMOUFLAGE EAR M
6	2010-12-07	537626	85232D	3	2010-12-07 14:57:00 UTC	4.95	12347	Iceland	SET/3 DECOUPAGE S
7	2010-12-07	537626	84997B	6	2010-12-07 14:57:00 UTC	3.75	12347	Iceland	RED 3 PIECE RETROS

- **가장 최근 구매 일자를 MAX() 함수로 찾아보기**

```
SELECT
   MAX(InvoiceDate) AS most_recent_date
FROM `project-c2dbf631-206a-40e8-a71`.modulabs_project.data;
```

행	most_recent_date
1	2011-12-09 12:50:00 UTC

- **유저 별로 가장 큰 InvoiceDay를 찾아서 가장 최근 구매일로 저장하기**

```
SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
FROM `project-c2dbf631-206a-40e8-a71`.modulabs_project.data
GROUP BY CustomerID;
```

행	CustomerID	InvoiceDay
1	12346	2011-01-18
2	12347	2011-12-07
3	12348	2011-09-25
4	12349	2011-11-21
5	12350	2011-02-02
6	12352	2011-11-03
7	12353	2011-05-19
8	12354	2011-04-21
9	12355	2011-05-09

- **가장 최근 일자(`most_recent_date`)와 유저별 마지막 구매일(`InvoiceDay`)간의 차이를 계산하기**

```
SELECT
  CustomerID,
  DATE_DIFF(MAX(InvoiceDay) OVER (), InvoiceDay, DAY) AS
recency
FROM (
  SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
  FROM `project-c2dbf631-206a-40e8-a71`.modulabs_projec
t.data
  GROUP BY CustomerID
);
```

행	CustomerID	recency
1	12346	325
2	12347	2
3	12348	75
4	12349	18
5	12350	310
6	12352	36
7	12353	204
8	12354	232
9	12355	214
10	12356	22
11	12357	33
12	12358	1
13	12359	7

- **최종 데이터 셋에 필요한 데이터들을 각각 정제해서 이어붙이고 지금까지의 결과를 `user_r` 이라는 이름의 테이블로 저장하기**

```
CREATE OR REPLACE TABLE `project-c2dbf631-206a-40e8-a71`
.modulabs_project.user_r AS
WITH user_last_purchase AS (
  SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
  FROM `project-c2dbf631-206a-40e8-a71`
.modulabs_project.data
  GROUP BY CustomerID
)

SELECT
  CustomerID,
  DATE_DIFF(MAX(InvoiceDay) OVER (), InvoiceDay, DAY) AS
recency
FROM user_last_purchase;
```

이 문으로 이름이 user_r인 새 테이블이 생성되었습니다.

행	CustomerID	recency
1	12985	0
2	16446	0
3	13777	0
4	17581	0
5	12748	0
6	15311	0

Frequency

- **고객마다 고유한 InvoiceNo의 수를 세어보기**

```
SELECT
  CustomerID,
  COUNT(DISTINCT InvoiceNo) AS purchase_cnt
FROM `project-c2dbf631-206a-40e8-a71`.modulabs_project.data
GROUP BY CustomerID;
```

행	CustomerID	purchase_cnt
1	12346	2
2	12347	7
3	12348	4
4	12349	1
5	12350	1
6	12352	8
7	12353	1
8	12354	1
9	12355	1
10	12356	3

- **각 고객 별로 구매한 아이템의 총 수량 더하기**

```
SELECT
  CustomerID,
  SUM(Quantity) AS item_cnt
FROM `project-c2dbf631-206a-40e8-a71`.modulabs_project.data
GROUP BY CustomerID;
```

행	CustomerID	item_cnt
1	12346	0
2	12347	2458
3	12348	2332
4	12349	630
5	12350	196
6	12352	463
7	12353	20
8	12354	530
9	12355	240
10	12356	1573

- **전체 거래 건수 계산과 구매한 아이템의 총 수량 계산의 결과를 합쳐서 `user_rf` 라는 이름의 테이블에 저장하기**

```
CREATE OR REPLACE TABLE project_name.modulabs_project.user_rf AS

-- (1) 전체 거래 건수 계산
WITH purchase_cnt AS (
  # [[YOUR QUERY]]
),

-- (2) 구매한 아이템 총 수량 계산
```

```
item_cnt AS (
  # [[YOUR QUERY]]
)

-- 기존의 user_r에 (1)과 (2)를 통합
SELECT
  pc.CustomerID,
  pc.purchase_cnt,
  ic.item_cnt,
  ur.recency
FROM purchase_cnt AS pc
JOIN item_cnt AS ic
  ON pc.CustomerID = ic.CustomerID
JOIN project_name.modulabs_project.user_r AS ur
  ON pc.CustomerID = ur.CustomerID;
```

이 문으로 이름이 user_rf인 새 테이블이 생성되었습니다.

행	CustomerID	purchase_cnt	item_cnt	recency
1	12713	1	505	0
2	13298	1	96	1
3	14569	1	79	1
4	13436	1	76	1
5	15520	1	314	1
6	15195	1	1404	2

Monetary

- **고객별 총 지출액 계산 (소수점 첫째 자리에서 반올림)**

```
SELECT
  CustomerID,
  ROUND(SUM(Quantity * UnitPrice), 1) AS user_total
FROM `project-c2dbf631-206a-40e8-a71`.modulabs_project.d
ata
GROUP BY CustomerID;
```

행	CustomerID	user_total
1	12346	0.0
2	12347	4310.0
3	12348	1437.2
4	12349	1457.5
5	12350	294.4
6	12352	1265.4
7	12353	89.0
8	12354	1079.4
9	12355	459.4
10	12356	2487.4

- **고객별 평균 거래 금액 계산**
  - **고객별 평균 거래 금액을 구하기 위해 1) `data` 테이블을 `user_rf` 테이블과 조인 (LEFT JOIN) 한 후, 2) `purchase_cnt` 로 나누어서 3) `user_rfm` 테이블로 저장하기**

```
CREATE OR REPLACE TABLE `project-c2dbf631-206a-40e8-a71
`.modulabs_project.user_rfm AS
SELECT
  rf.CustomerID AS CustomerID,
  rf.purchase_cnt,
  rf.item_cnt,
  rf.recency,
  ut.user_total,
  ROUND(ut.user_total / rf.purchase_cnt, 1) AS user_aver
age
FROM `project-c2dbf631-206a-40e8-a71`.modulabs_project.u
ser_rf rf
LEFT JOIN (
  -- 고객 별 총 지출액
  SELECT
    CustomerID,
    ROUND(SUM(Quantity * UnitPrice), 1) AS user_total
  FROM `project-c2dbf631-206a-40e8-a71`.modulabs_projec
```

```
t.data
  GROUP BY CustomerID
) ut
ON rf.CustomerID = ut.CustomerID;
```

이 문으로 이름이 user_rfm인 새 테이블이 생성되었습니다.

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average
1	12713	1	505	0	794.5	794.5
2	13298	1	96	1	360.0	360.0
3	14569	1	79	1	227.4	227.4
4	15520	1	314	1	343.5	343.5
5	13436	1	76	1	196.9	196.9
6	14204	1	72	2	150.6	150.6

RFM 통합 테이블 출력하기

- **최종 user_rfm 테이블을 출력하기**

```
SELECT *
FROM `project-c2dbf631-206a-40e8-a71`.modulabs_project.u
ser_rfm;
```

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average
1	12713	1	505	0	794.5	794.5
2	13298	1	96	1	360.0	360.0
3	14569	1	79	1	227.4	227.4
4	15520	1	314	1	343.5	343.5
5	13436	1	76	1	196.9	196.9
6	14204	1	72	2	150.6	150.6
7	15195	1	1404	2	3861.0	3861.0
8	15471	1	256	2	454.5	454.5
9	12650	1	250	3	242.4	242.4
10	16569	1	93	3	124.2	124.2
11	12442	1	181	3	144.1	144.1
12	15992	1	17	3	42.0	42.0

11-8. 추가 Feature 추출

1. 구매하는 제품의 다양성

- 1) 고객 별로 구매한 상품들의 고유한 수를 계산하기
  2) `user_rfm` 테이블과 결과를 합치기
  3) `user_data` 라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE `project-c2dbf631-206a-40e8-a71`.mo
dulabs_project.user_data AS
WITH unique_products AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT StockCode) AS unique_products
  FROM `project-c2dbf631-206a-40e8-a71`.modulabs_project.da
ta
  GROUP BY CustomerID
)

SELECT
  ur.*,
  up.* EXCEPT (CustomerID)
FROM `project-c2dbf631-206a-40e8-a71`.modulabs_project.user
_rfm AS ur
JOIN unique_products AS up
ON ur.CustomerID = up.CustomerID;
```

이 문으로 이름이 user_data인 새 테이블이 생성되었습니다.

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products
1	14705	1	100	198	179.0	179.0	1
2	13747	1	8	373	79.6	79.6	1
3	13135	1	4300	196	3096.0	3096.0	1
4	18233	1	4	325	440.0	440.0	1
5	16078	1	16	283	79.2	79.2	1
6	18141	1	-12	360	-35.4	-35.4	1

2. 평균 구매 주기

- **고객들의 쇼핑 패턴을 이해하는 것을 목표 (고객 별 재방문 주기 살펴보기)**
  - **평균 구매 소요 일수를 계산하고, 그 결과를 `user_data` 에 통합**

```
CREATE OR REPLACE TABLE project_name.modulabs_project.user_
data AS
WITH purchase_intervals AS (
  -- (2) 고객 별 구매와 구매 사이의 평균 소요 일수
  SELECT
    CustomerID,
    CASE WHEN ROUND(AVG(interval_), 2) IS NULL THEN 0 ELSE
ROUND(AVG(interval_), 2) END AS average_interval
  FROM (
    -- (1) 구매와 구매 사이에 소요된 일수
    SELECT
      CustomerID,
      DATE_DIFF(InvoiceDate, LAG(InvoiceDate) OVER (PARTITI
ON BY CustomerID ORDER BY InvoiceDate), DAY) AS interval_
    FROM
      project_name.modulabs_project.data
    WHERE CustomerID IS NOT NULL
  )
  GROUP BY CustomerID
)

SELECT u.*, pi.* EXCEPT (CustomerID)
FROM project_name.modulabs_project.user_data AS u
LEFT JOIN purchase_intervals AS pi
ON u.CustomerID = pi.CustomerID;
```

이 문으로 이름이 user_data인 테이블이 교체되었습니다.

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	average_interval
1	16765	1	4	294	34.0	34.0	1	0.0
2	15562	1	39	351	134.6	134.6	1	0.0
3	16093	1	20	106	17.0	17.0	1	0.0
4	12603	1	56	21	613.2	613.2	1	0.0
5	14424	1	48	17	322.1	322.1	1	0.0
6	16428	1	-1	81	-3.0	-3.0	1	0.0

3. 구매 취소 경향성

- **고객의 취소 패턴 파악하기**
  1) 취소 빈도(cancel_frequency) : 고객 별로 취소한 거래의 총 횟수
  2) 취소 비율(cancel_rate) : 각 고객이 한 모든 거래 중에서 취소를 한 거래의 비율
    - **취소 빈도와 취소 비율을 계산하고 그 결과를 `user_data` 에 통합하기**
      (취소 비율은 소수점 두번째 자리)

```
CREATE OR REPLACE TABLE project_name.modulabs_project.user_
data AS

WITH TransactionInfo AS (
  SELECT
    CustomerID,
    # [[YOUR QUERY]] AS total_transactions,
    # [[YOUR QUERY]] AS cancel_frequency
  FROM project_name.modulabs_project.data
  # [[YOUR QUERY]]
)

SELECT u.*, t.* EXCEPT(CustomerID), # [[YOUR QUERY]] AS can
cel_rate
FROM `project_name.modulabs_project.user_data` AS u
LEFT JOIN TransactionInfo AS t
ON # [[YOUR QUERY]];
```

이 문으로 이름이 user_data인 테이블이 교체되었습니다.

- **다양한 컬럼들을 활용하여 고객의 구매 패턴과 선호도를 보다 심층적으로 이해할 수 있도록 최종적으로 `user_data` 를 출력하기**

```
SELECT *
FROM `project-c2dbf631-206a-40e8-a71`.modulabs_project.user
_data
LIMIT 20;
```

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	average_interval	total_transactions	cancel_frequency	cancel_rate
1	16252	1	-158	366	-295.1	-295.1	21	0.0	1	21	21.0
2	15823	1	-282	337	-811.9	-811.9	15	0.0	1	15	15.0
3	12536	2	2672	7	4151.3	2075.7	259	0.19	2	9	4.5
4	12607	2	0	58	0.0	0.0	101	0.01	2	101	50.5
5	14557	2	0	64	0.0	0.0	16	0.68	2	16	8.0
6	12454	2	0	53	0.0	0.0	15	0.1	2	15	7.5
7	18072	2	48	155	12.2	6.1	19	0.91	2	16	8.0
8	14205	3	100	16	228.5	76.2	34	9.43	3	14	4.67
9	12967	4	243	3	1194.8	298.7	21	11.5	4	14	3.5

회고

[회고 내용을 작성해주세요]

Keep :

Problem :

Try :